Server side TLS:

Tools used :

- `Keytool`
- `Openssl`
- [mqtt]

Create a server side JKS

```
keytool -genkey -keyalg RSA -alias hivemq -keystore hivemq.jks \
 -storepass password -validity 360 -keysize 2048
```

Export a pem file from the server keystore

```
keytool -exportcert -keystore hivemq.jks -alias hivemq -keypass password \
 -storepass password -rfc -file hivemq-server-cert.pem
```

Client side TLS:

```
openssl req -x509 -newkey rsa:2048 -nodes -days 360 \
 -keyout mqtt-client-key.pem \
 -out mqtt-client-cert.pem
```

(set common name (CN) to start with CAR followed by 10 numbers)

Convert to crt

```
openssl x509 -outform der -in mqtt-client-cert.pem -out mqtt-client-cert.crt
```

Add crt to trusted client keystore:

```
keytool -import -file mqtt-client-cert.crt -alias client \
-keystore trust-store.jks -storepass password -noprompt
```

Setup the broker for mTLS

Copy both JKS files (`trust-store.jks and hivemq.jks`) to the HiveMQ base directory. After that rewrite conf/config.xml as shown below and restart the broker.

```
   <tls-tcp-listener>
           <port>8883</port>
           <bind-address>0.0.0.0</bind-address>
           <tls>
               <keystore>
                   <path>hivemq.jks</path>
                   <password>password</password>
                   <private-key-password>password</private-key-password>
               </keystore>

               <truststore>
                   <path>trust-store.jks</path>
                   <password>password</password>
               </truststore>

               <client-authentication-mode>REQUIRED</client-authentication-mode>
           </tls>
       </tls-tcp-listener>
```

test:

when "Unable to connect. Reason: `'No subject alternative names present'` , than add nissanbroker to the local hostfile (/etc/hosts)

```
127.0.0.1        localhost localbroker broker nissanbroker
```

Nwt test with :

```
mqtt pub -t test -p 8883 --cert mqtt-client-cert.pem --key mqtt-client-key.pem \
--cafile hivemq-server-cert.pem \
-m testtext \
-h nissanbroker
```

Implement X.509 extraction to assign roles

Add roles to the hivemq-4.35.0/extensions/hivemq-enterprise-security-extension/conf/ese-file-realm.xml file:

```
       <!-- roles are fetched via AUTHENTICATION_ROLE_KEY /
authorization-role-key-->
       <roles>
       <role>
               <id>ADM</id>
               <permissions>
                   <permission>
                       <topic>#</topic>
                       <activity>ALL</activity>
                       <qos>ALL</qos>
                   </permission>
               </permissions>
           </role>
           <role>
               <id>CAR</id>
               <permissions>
                   <permission>
                       <topic>devices/${mqtt-clientid}/#</topic>
                       <activity>ALL</activity>
                       <qos>ALL</qos>
                   </permission>
               </permissions>
           </role>
       </roles>
```

Notice that the CAR role enforces the use of the MQTT client ID as part of the topic structure:

```
 <topic>devices/${mqtt-clientid}/#</topic>
```

Setup a security pipeline

Add hivemq-4.35.0/extensions/hivemq-enterprise-security-extension/conf/config.xml

```
  <realms>
       <file-realm>
           <name>file-realm</name>
           <enabled>true</enabled>
           <configuration>
               <file-path>conf/ese-file-realm.xml</file-path>
           </configuration>
       </file-realm>
   </realms>

   <pipelines>
       <!-- secure access to the mqtt broker -->
       <listener-pipeline listener="ALL">
           <allow-all-authentication-manager/>

           <!-- setting the role name basd o regex on client ID -->
           <authorization-preprocessors>
               <logging-preprocessor>
                   <message>1-authorization pre: The content of the mqtt-clientid
ESE-Variable: ${mqtt-clientid} </message>
                   <level>debug</level>
                   <name>com.example.logger</name>
               </logging-preprocessor>

                <x509-preprocessor prefix="{{" postfix="}}">
                   <x509-extractions>
                       <x509-extraction>
                           <x509-field>subject-common-name</x509-field>
                           <ese-variable>authorization-role-key</ese-variable>
                       </x509-extraction>
                   </x509-extractions>
               </x509-preprocessor>

               <logging-preprocessor>
                   <message>2-authorization pre: The content of the
authorization-role-key ESE-Variable: ${authorization-role-key} </message>
                   <level>debug</level>
                   <name>com.example.logger</name>
               </logging-preprocessor>

               <regex-preprocessor>
                   <pattern>((.{3})(.{10}))</pattern>
                   <group>2</group>
                   <variable>authorization-role-key</variable>
               </regex-preprocessor>


               <logging-preprocessor>
                   <message>3-authorization pre: The content of the
authorization-role-key ESE-Variable: ${authorization-role-key} </message>
                   <level>debug</level>
                   <name>com.example.logger</name>
               </logging-preprocessor>
```

```
          </authorization-preprocessors>

          <!-- authorize over a file -->
          <file-authorization-manager>
              <realm>file-realm</realm>
              <use-authorization-key>false</use-authorization-key>
              <use-authorization-role-key>true</use-authorization-role-key>
          </file-authorization-manager>
      </listener-pipeline>
   </pipelines>

</enterprise-security-extension>
```

Notice that authentication is fully skipped since we consider the contents of the client certificate trustworthy.

Besides some logging the main process loads the x.509 subject-common-name into the ese-variable authorization-role-key.

This authorization-role-key later processed with a regex so it renders only containing the first 3 characters (CAR of ADM).

```
              <regex-preprocessor>
                  <pattern>((.{3})(.{10}))</pattern>
                  <group>2</group>
                  <variable>authorization-role-key</variable>
              </regex-preprocessor>
```

These three chars are matched to the role defined in ese-file-realm.xml file so the correct role permissions are granted.

Test:

The leading part of the CN determines the role and though this the effective permissions:

```
          <role>
               <id>CAR</id>
```

Check CN:

```
openssl x509 -noout -subject -in mqtt-client-cert.pem
>> subject=C=NL, ST=OV, L=Ens, O=IT, OU=IT, CN=CAR1234567890
```

Topic must match MQTT client-ID:

```
 <topic>devices/${mqtt-clientid}/#</topic>
```

CN determines the role/perms:

```
mqtt pub -p 8883 --cert mqtt-client-cert.pem --key mqtt-client-key.pem \
--cafile hivemq-server-cert.pem \
-m 120kmh \
-h nissanbroker \
--identifier=VIN0123456789 \
--topic devices/VIN0123456789/speed
```

Comment more than welcome,

Kamiel